

remez

a digital filter designer

remez designs digital filters and shows you what you got. You type a specification on the left; it designs the filter as you type and plots the result on the right. When you like it, you export the coefficients, C, RTL, or the design itself.

Two families are on offer:

- **FIR**, by the Parks–McClellan (Remez) exchange. Linear phase, unconditionally stable, as many taps as it takes.
- **IIR**, by the bilinear transform of a classical analogue prototype. Far fewer coefficients for the same selectivity, at the cost of non-linear phase and the need to watch stability.

Every figure in this document is generated from the program itself by `test/make_docs_test.dart`, so if the interface changes and the figures are not regenerated, the tests notice. That generator loads real fonts before rendering: `flutter test` normally substitutes a font whose every glyph is a filled rectangle, which is right for a layout golden and useless for a screenshot.

Contents

1	The five-minute version	3
2	Reading the plots	3
2.1	Lowpass	3
2.2	Highpass	4
2.3	Bandpass	4

3	The panels, one at a time	6
3.1	Mode	6
3.2	Filter — FIR	6
3.3	Bands and constraints — FIR	7
3.4	Filter — IIR	9
3.5	Bands and specification — IIR	9
3.6	Arithmetic	10
3.7	File	13
3.8	Display	14
3.9	Signal	15
3.10	Result	16
4	Filter types and what they are good for	17
4.1	The four linear-phase FIR types	17
4.2	Three ways to design an FIR	17
4.3	Half band and polyphase	18
4.4	FIR against IIR	19
4.5	The four IIR approximations	19
5	The save file	20
5.1	A complete example	20
5.2	Semantics	21
5.3	Compatibility	22

1 The five-minute version

1. Leave **Mode** on *FIR*.
2. In **Filter**, set **Sample rate** to whatever your system runs at. The band edges rescale with it, so you can work in Hz rather than fractions.
3. In **Bands and constraints**, set the passband and stopband edges.
4. Watch the top plot. If the stopband is not deep enough, raise **Taps**.
5. **File** → **Save coefficients...** for a CSV or a C header, or **Save C...** for a filter you can compile and run.

Everything else in this document is detail.

2 Reading the plots

2.1 Lowpass

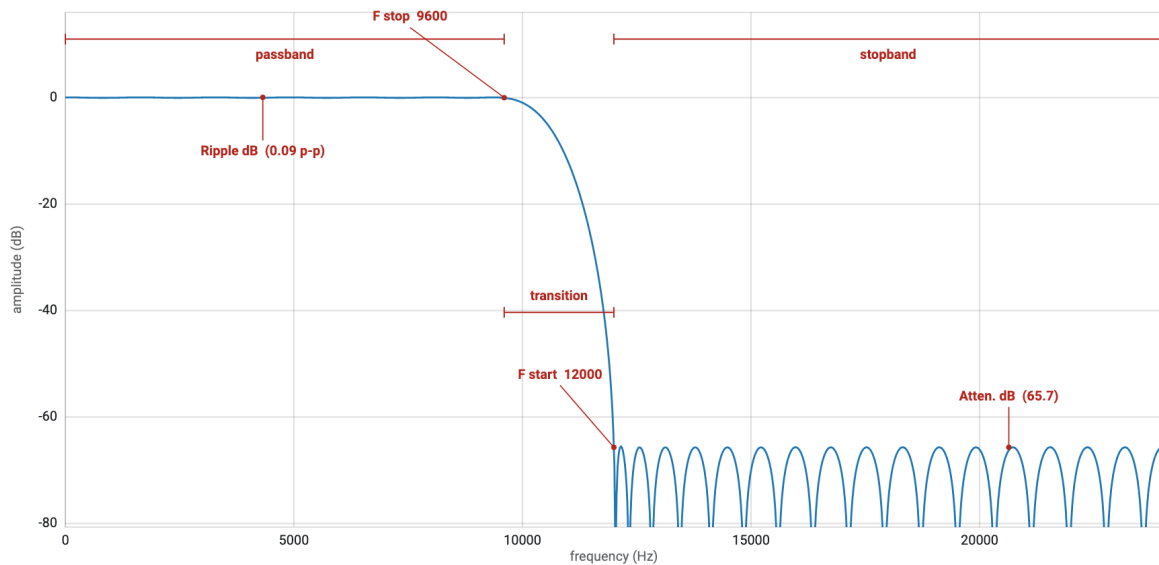


Figure 1: A lowpass response, with each parameter named where it acts on the curve.

The band edges you type are the ends of the flat regions, *not* the -3 dB point or the middle of the skirt. The *transition* between them is empty: nothing is specified there and the design is free to do as it likes, which is why a narrower transition costs taps.

- **F stop** of the passband row sets where the passband ends.
- **F start** of the stopband row sets where the stopband begins.
- **Ripple dB** is the peak-to-peak wobble allowed across the passband.
- **Atten. dB** is how far down the stopband is held.

The ripple is equal across the whole band and the stopband lobes are all the same height. That is what *equiripple* means, and it is the point of the Remez exchange: for a given number of taps, no filter has a smaller worst-case error. The red dots on the curve in the app are the extremal frequencies — the points where the error touches its limit.

2.2 Highpass

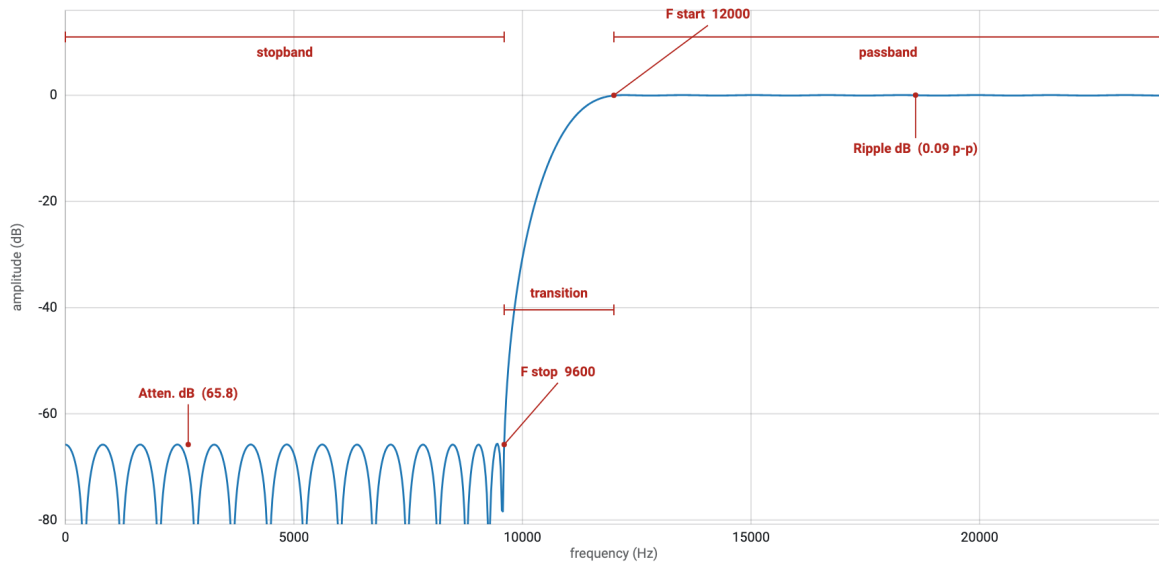


Figure 2: A highpass. The same picture reflected: the stopband row comes first and the passband row second.

A highpass wants an *odd* number of taps if it must reach all the way to Nyquist: an even-length symmetric filter (type II) is forced to zero there and cannot be a highpass. `remez` will tell you in the **Result** panel if you have asked for something the type cannot do.

2.3 Bandpass

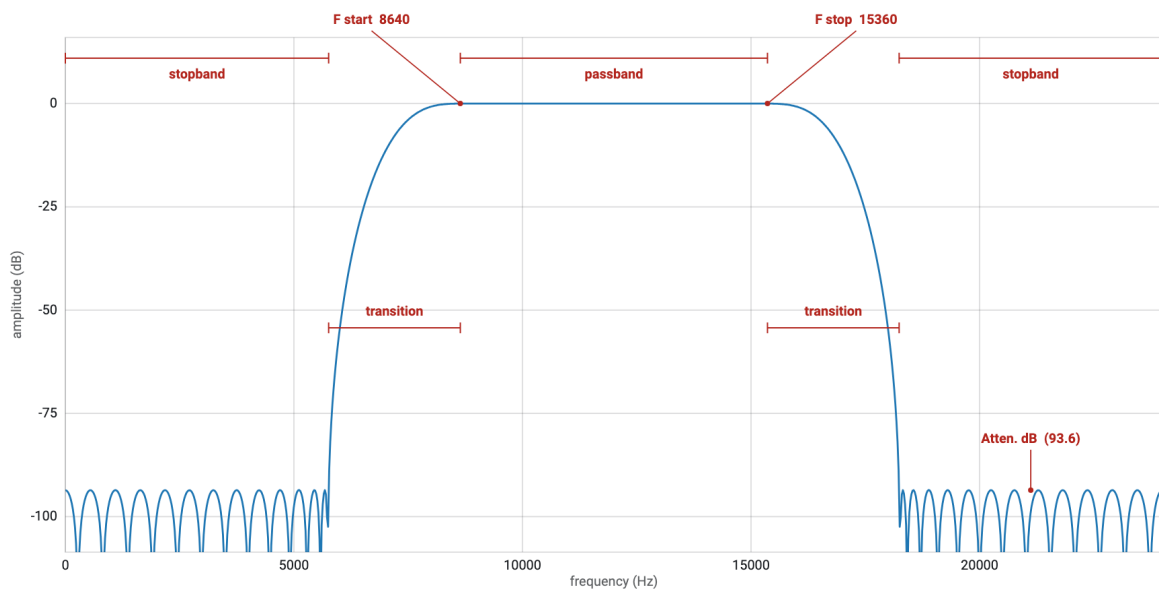


Figure 3: A bandpass: three bands, two transitions, and they cost separately — the narrower one sets the taps.

The two stopbands need not have the same attenuation. Give them different **Weight** values (or different **Spec (dB)**) and the exchange will hold one tighter than the other, spending its taps

where you asked.

3 The panels, one at a time

3.1 Mode

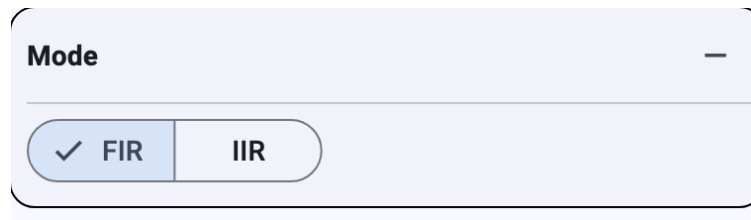


Figure 4: The Mode panel.

FIR or IIR. The two have different **Filter** and band panels; everything else is shared. Both designs are kept, so switching back and forth does not lose what you typed.

3.2 Filter — FIR

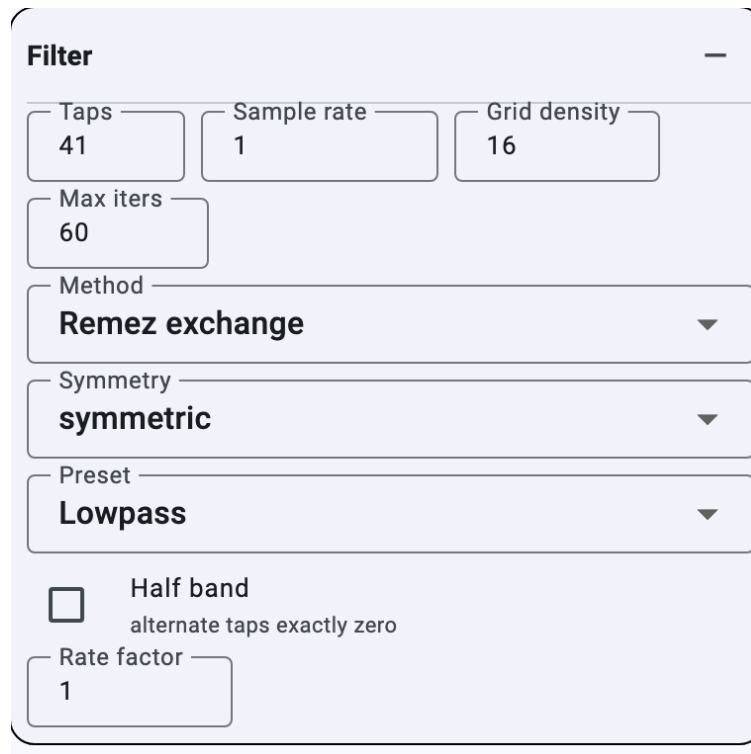


Figure 5: The FIR Filter panel.

field	what it does
Taps	The filter length N . More taps buy a narrower transition or a deeper stopband, at a cost of N multiplies and $N - 1$ delays per sample.
Sample rate	The units everything else is in. Changing it <i>rescales</i> the band edges so the filter stays the same — set it first, then type edges in Hz.
Grid density	How finely the exchange samples each band when it searches for the error peaks, in points per coefficient. 16 is the usual value. Too coarse and the design steps over a ripple peak and reports a smaller δ than it achieved; see below.
Max iters	The iteration cap. 60 is far more than a converging design needs; if the Result panel says <i>did not converge</i> , the design ran out.
Symmetry	<code>symmetric</code> for ordinary filters; <code>antisymmetric</code> for a Hilbert transformer or a differentiator, which need a 90° phase shift.
Method	Which algorithm designs the filter: the Remez exchange, weighted least squares, or the window method. See section 4.2.
Window	Only with the window method: which taper cuts the ideal response, and so how deep the stopband gets.
Kaiser beta	Only with the Kaiser window, whose attenuation is set by this number rather than fixed.
Preset	Loads a worked example into the band table. A good starting point for a new design.
Half band	Design a half-band lowpass: alternate taps come out exactly zero, so half the multiplies disappear. See section 4.3.
Passband edge	Only with Half band : the one edge you get to choose. The stopband starts at its mirror about a quarter of the sample rate.
Rate factor	Split the filter into this many polyphase components, for a decimator or an interpolator. 1 means no rate change.

On grid density. The reported δ is a *lower bound* on the true deviation, never an upper one. Measured on the default lowpass against the response evaluated at 200 001 points:

grid density	reported δ	true peak error	optimism
4	0.02848	0.03029	+6.3 %
16 (default)	0.02877	0.02901	+0.8 %
64	0.02884	0.02886	+0.04 %

Leave it at 16 while exploring; raise it to 64 before you trust the number or ship the coefficients.

3.3 Bands and constraints — FIR

Bands are given in order and must not overlap; the gaps between them are the transition regions.

+

–

F start

0

F stop

0.2

D at start

1

D at stop

1

×

Weight

1

Spec (dB)

0.5

☐ 1/f

F start

0.25

F stop

0.5

D at start

0

D at stop

0

×

Weight

10

Spec (dB)

50

☐ 1/f

☐

Weights from the Spec column

passband ripple dB p-p / stopband atten. dB

Figure 6: The FIR band table, one row per band.

column	what it does
F start, F stop	The band edges, in the sample-rate units.
D at start, D at stop	The desired amplitude at each end. Equal for a flat band; different for a sloped one — a raised-cosine skirt, or a differentiator’s $2\pi f$.
Weight	How hard this band is held relative to the others. The exchange equalises $W D - A $, so doubling a weight halves that band’s error and spends the taps elsewhere. Only the ratios matter.
Spec (dB)	An alternative to Weight , in decibels: peak-to-peak ripple for a band with a non-zero target, attenuation for one that targets zero.
1/f	Weight the band as w/f rather than a constant, which equalises <i>relative</i> error. What a differentiator needs — a fixed absolute error is negligible at the top of the band and hopeless at the bottom.
×	Remove the row.

The $\oplus$ button in the panel header adds a band.

Weights from the Spec column. Tick it and the **Weight** column greys out; the weights are derived from the dB figures instead. This is usually what you want, because “0.5 dB ripple and 60 dB attenuation” is a specification and “weights 1 and 10” is a guess at one. The **Result** panel then checks each band against its spec and, if one is missed, estimates the taps that would meet it.

Filter

Response
bandpass

Approximation
butterworth

☒ Smallest order

Order
5

Sample rate
1

Figure 7: The IIR Filter panel.

3.4 Filter — IIR

field	what it does
Response	Lowpass, Highpass, Bandpass or Bandstop. Changing it brings edge values that make sense for the new shape.
Approximation	Butterworth, Chebyshev I, Chebyshev II or Elliptic; see section 4.5.
Order	The prototype order. The digital filter has twice this many poles for a bandpass or bandstop.
Smallest order	Let remez pick the lowest order that meets the specification. Usually leave this on; turn it off to explore what a particular order does.

3.5 Bands and specification — IIR

Bands and specification

Passband 1
0.2

Passband 2
0.3

Stopband 1
0.12

Stopband 2
0.38

Ripple dB
0.5

Atten. dB
40

Figure 8: The IIR band panel: edges and tolerances rather than a table.

field	what it does
Passband (1, 2)	Where the passband ends. Two fields for a bandpass or bandstop.
Stopband (1, 2)	Where the stopband begins.
Ripple dB	Peak-to-peak passband ripple. Butterworth has no ripple by construction, but the figure is still used: the prototype is widened so the response is exactly this far down at the passband edge, rather than at the conventional -3 dB point. That puts all four approximations on the same footing.
Atten. dB	Required stopband attenuation.

The design places the edge that its approximation actually pins down: the passband edge for Butterworth, Chebyshev I and elliptic, the stopband edge for Chebyshev II. The other edge falls where the order puts it.

3.6 Arithmetic

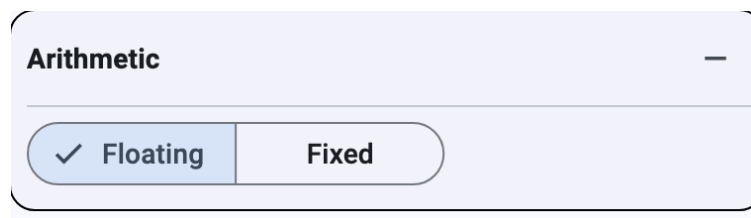


Figure 9: In floating point the coefficients are doubles and the plots show the ideal design.

Switch to **Fixed** and remez quantizes the coefficients and re-analyses, so the plots and the report show the filter you would actually build.

Arithmetic

Floating

✓ Fixed

Word bits

12

Headroom

2

☒ Place the binary point automatically

Q-1.12 step 2.441e-4

Hardware

Structure

chain

☐ Fold the symmetric taps
one multiplier per pair

☒ Coefficients as constants
otherwise a runtime port

☒ Write a testbench too

☐ Measure the arithmetic noise
slow: runs the datapath

☐ Shade the coefficient sensitivity
what $\pm\frac{1}{2}$ LSB of error covers

Figure 10: In fixed point the word length and the hardware options appear.

field	what it does
Word bits	Coefficient word length.
Headroom	Extra integer bits in the datapath, above the coefficient format. This is what keeps the adders off their limits; 2 is a sensible default.
Place the binary point automatically	Puts the binary point where the largest coefficient just fits. Turn it off to set the fraction length yourself.
Module name	What the generated module is called, and with it the two files it goes in. It used to be taken from the name typed into the save dialog; the RTL export asks for a folder now, so the name is a setting like any other and is saved with the design. Anything not usable as an identifier is folded to lower case with underscores.
Structure	How the multiplies are summed in hardware: chain (one adder per tap — smallest, slowest), tree (balanced, registered between levels — fastest), mac (one multiplier reused, one term per clock — smallest of all, one sample per N clocks).
Fold the symmetric taps	A linear-phase filter has equal taps in pairs, so one pre-add plus one multiply serves two of them. Roughly halves the multipliers.
Coefficients as constants	Compile them into the RTL so synthesis can specialise each multiplier. Turn it off for a runtime coefficient port.
Write a testbench too	Emit a self-checking testbench beside the RTL. It carries the expected output of every sample, so running it proves the hardware matches the plots.

The line under the fields reports the Q format, the step size, and a warning if any coefficient saturated. A saturated coefficient means the hardware is not the filter you designed — give it more bits.

The last two boxes are analyses rather than build options, and both cost real time, which is why they are off by default.

Measure the arithmetic noise. Rounding the coefficients is only half of what fixed point does to a filter; the other half is that every product and every sum is rounded too. This drives the exact integer datapath — the one the RTL export would generate, structure and folding included — with a few thousand samples, compares its output against the same filter computed exactly, and takes the spectrum of the difference. That level is drawn on the magnitude plot as a dashed line, and the **Result** panel says what it means: a stopband below the line is a number on a plot rather than a filter you can build.

Shade the coefficient sensitivity. Nudges every coefficient independently by up to half an LSB, a hundred and twenty-odd times, and shades the region the responses cover. A thin band means the design has margin. A band that swallows the stopband means the filter only works at

the exact values it was given. For an IIR it also counts the draws that came out unstable, which is the failure that rounding a high-order elliptic design really produces.

3.7 File

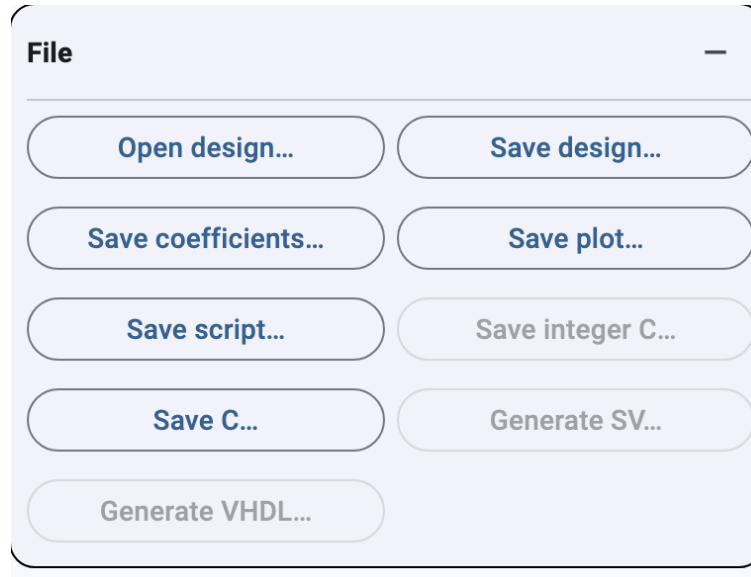


Figure 11: The File panel.

button	what it writes
Open design...	Reads a <code>.remz</code> (or an older <code>.json</code>) design.
Save design...	Writes the whole specification as <code>.remz</code> ; see section 5.
Save coefficients...	The coefficients alone. A <code>.h</code> extension gets C arrays; anything else gets CSV with a commented header. In fixed point both carry the stored integers alongside the rounded values.
Save plot...	The right-hand pane as a PNG — whichever view is showing, in whichever theme, at twice screen resolution.
Save C...	One self-contained C file: a library (<code>init_filter</code> / <code>process_sample</code> / <code>free_filter</code>) and a program that filters raw 64-bit floats from stdin to stdout.
Save script...	A NumPy module or a MATLAB function, chosen by the extension you give it — <code>.py</code> or <code>.m</code> . Both define the coefficients and call the library routine; an FIR goes to <code>lfilter/filter</code> , a cascade to <code>sosfilt</code> .
Save integer C...	The same filter with no floating point anywhere: signed integers, products rounded and saturated, sums saturated, in the order the chosen structure sums them. It is the <i>same</i> arithmetic as the generated RTL, bit for bit, which makes it the reference model to run on the target before the hardware exists. Needs fixed point.
Generate SV..., Generate VHDL...	Synthesisable RTL for the structure chosen in Arithmetic , with its testbench. These two ask for a <i>folder</i> rather than a file name, because they write a pair of files — <code><module>.sv</code> and <code><module>_tb.sv</code> — and a save dialog on a sandboxed desktop grants only the one file named in it, which left the testbench unwritable. The files are named after Module name in the Arithmetic panel. Both need fixed-point coefficients; until then they are disabled and the tooltip says why.

3.8 Display

Plot view is the response plots. **Design view** draws the filter as the thing you would build — the adders, delays and multipliers, each labelled with its actual constant. **Magnitude in dB** switches the top plot between decibels and linear amplitude. **Auto / Light / Dark** follows the desktop or forces a theme; the saved plot uses whichever is showing.

Three checkboxes choose which of the optional plots are drawn:

- **Group delay** — how many samples each frequency is held up by. A linear-phase FIR draws

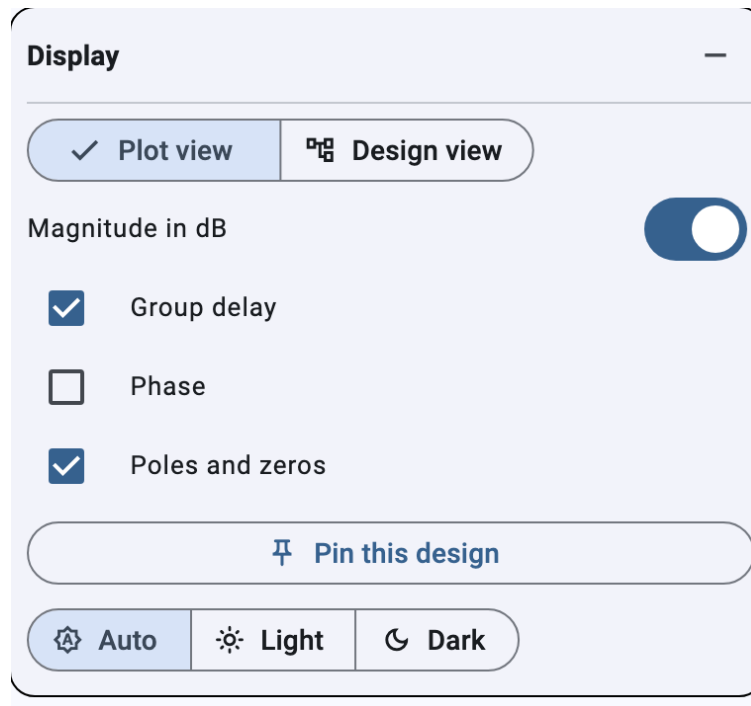


Figure 12: The Display panel.

a flat line at $(N - 1)/2$, which is the claim being made for it, verified rather than asserted. An IIR does not: its delay peaks around the band edges, and that peak is the price of the coefficients it saved.

- **Phase** — the same information one integral earlier, unwrapped. Off by default because group delay says it more directly. The steps of 180° in an FIR’s phase are its zeros on the unit circle.
- **Poles and zeros** — the z plane, with the unit circle drawn. Every pole inside it or the filter is unstable, and in fixed point the design’s poles and the poles the *rounded* coefficients actually give are drawn together, so a pole that rounding has pushed onto the circle is something you see rather than something you infer from a magnitude plot behaving oddly.

Pin this design keeps the current response on the axes, dashed, while you edit another against it — the way to answer “is 61 taps really buying me anything here?” without alt-tabbing between screenshots. The button turns into **Unpin** and names what it is holding.

The two arrows in the title bar are undo and redo, on **Cmd-Z** and **Shift-Cmd-Z**. They step through complete designs, not individual keystrokes, and reach everything the save file reaches.

3.9 Signal

Push a test signal through the filter and plot what came out, alongside what went in. The response plots say what happens to a sine wave of every frequency, one at a time and forever; that is the complete answer and it is not always the legible one.

signal	what it is for
impulse	the taps themselves
step	overshoot, ringing, and the pre-echo linear phase pays for its symmetry
chirp	the whole band swept, so the transition is a fade you can point at
tone	one frequency, for gain and delay
noise	everything at once, for the noise floor
square	harmonics, and what happens to the ones the filter removes

Samples is how long a run to make, from 16 to 8192. Ask for something outside that and it refuses and says so, rather than quietly running a different number — being handed 8192 samples when you asked for 100 000, with nothing said, is how a measurement gets misread.

In fixed point it runs the signal twice: once through the design in double precision and once through the exact integer datapath. Both are drawn, and the title reports the difference between them and how many samples the datapath had to clip. Clipping is not rounding that averages away — it is the filter ceasing to be linear — so one clipped sample is a headroom problem, not a word-length one.

3.10 Result

Result	
type 1	(symmetric, odd length 41)
method	Remez exchange

Figure 13: The report: what was designed, and how well.

Filter type and length, iterations, the weighted δ , then a table of each band's achieved deviation in both linear and dB terms. In fixed point it adds the Q format, the worst rounding error and the datapath width; with **Weights from the Spec column** on it adds a spec check per band. It ends with the filter itself — the taps for an FIR, the second-order sections for an IIR — which you can select and copy.

4 Filter types and what they are good for

4.1 The four linear-phase FIR types

Symmetry and length together decide what a filter can do. `remez` picks the type for you and names it in the report; the constraints are not arbitrary but come from what the amplitude response is forced to be at DC and at Nyquist.

type	symmetry	length	zero at DC?	zero at Nyq.?	use for
I	symmetric	odd	no	no	anything: lowpass, highpass, bandpass, bandstop, multiband
II	symmetric	even	no	forced	lowpass and bandpass only; never a highpass
III	antisymmetric	odd	forced	forced	Hilbert transformers, differentiators
IV	antisymmetric	even	forced	no	differentiators, antisymmetric highpass work

Practically: leave **Symmetry** on `symmetric` and use an odd number of taps unless you have a reason not to. Choose `antisymmetric` when you want a 90° phase shift — the Hilbert transformer and differentiator presets set it for you.

4.2 Three ways to design an FIR

The **Method** pulldown offers three, and they are not ranked: each minimises something different, and which one is right depends on what your specification actually means.

method	minimises	the stopband it gives	iterates?
Remez exchange	the <i>worst</i> weighted error	one flat wall of equal lobes	yes
least squares	the <i>total</i> squared error	worst at the transition, quieter out	no
window	nothing	whatever the window's sidelobes are	no

The exchange is the default because a specification is usually a limit that must not be exceeded, and minimax is exactly the promise “nowhere worse than this”. For a given number of taps no filter has a smaller worst-case error.

Least squares gives that up and buys total energy instead. Its error is largest at the band edges and falls away from them, so the far stopband ends up much deeper than an equiripple design would put it. Reach for it when the thing being rejected is broadband, or when the filter feeds something that integrates: holding a corner nobody is standing on is taps spent for nothing.

The window method does not optimise at all. It takes the exact impulse response of the ideal brick wall, which is infinitely long, cuts it to length, and tapers the cut so the truncation does

not ring. Two things recommend it: there is no iteration to fail to converge, and the stopband depth is a property of the window rather than of the design.

window	sidelobes	notes
rectangular	≈ 21 dB	no taper at all; the ringing is the point of comparison
Hann	≈ 44 dB	sidelobes fall away fastest, so the far stopband is the best of the fixed three
Hamming	≈ 53 dB	lowest <i>first</i> sidelobe, but it rolls off slowly
Blackman	≈ 74 dB	the deepest, and the widest main lobe, so it needs the longest filter
Kaiser	set by β	the adjustable one: β trades transition width against depth

The catch is the other direction. A window only *reaches* its figure once the filter is long enough for the window's main lobe to fit inside the transition. Ask for a narrow transition with a Blackman window and a short filter and you get neither: 161 taps across this document's example transition reaches -75 dB, and 81 taps manages only -39 dB.

4.3 Half band and polyphase

Two ways of not doing arithmetic you do not have to do.

A half-band lowpass has every other tap at exactly zero. That is not a trick played on an ordinary design — it falls out of the problem. Ask for a filter that is 1 up to f_p and 0 from $f_s/2 - f_p$, with the two bands weighted *equally*, and the answer satisfies

$$A(\omega) + A(\pi - \omega) = 1$$

identically: if it did not, averaging it with its own mirror would give an equally good filter, and the minimax solution is unique. Written out in taps, that identity says the centre tap is $\frac{1}{2}$ and every tap an even number of places from it is zero. So ticking **Half band** does not post-process anything. It constrains: one band edge instead of a table, equal weights, and a length rounded to the nearest $4k + 3$, because those are the conditions under which the answer is a half-band filter at all. The exchange then leaves the vanishing taps at about 10^{-16} , and the program sets them to the zero they mathematically are. A 43-tap half band needs 23 multiplies instead of 43, and the **Result** panel says so.

A polyphase decomposition is what makes a rate change cost what it should. Decimating by M throws away $M - 1$ of every M outputs, so computing them was wasted; splitting the filter into M sub-filters, each fed every M th sample, and summing them computes only the outputs that survive. Interpolating by L is the mirror image. Either way the multiplies per sample fall by the rate factor and *no arithmetic changes* — the output is the same to the last bit. Set **Rate factor** above 1 and the phases appear in the **Result** panel and in the saved coefficients, as **e0**, **e1**, ... where phase p holds taps p , $p + M$, $p + 2M$ and so on.

4.4 FIR against IIR

	FIR	IIR
phase	exactly linear	not linear; group delay varies, worst near the band edges
stability	unconditional	must be checked; remez reports $\max p $
coefficients for a sharp filter	many (tens to hundreds)	few (a handful of biquads)
quantization	graceful; the response degrades smoothly	poles can move onto or outside the unit circle
transient	finite, N samples	infinite

Reach for FIR when phase matters, when the filter must be provably stable, or when it will run on hardware with multipliers to spare. Reach for IIR when you need a sharp filter cheaply and phase is not critical — audio equalisation, control loops, decimation front-ends.

4.5 The four IIR approximations

Everything gives up something to get selectivity.

	passband	stopband	selectivity	phase
Butterworth	maximally flat	monotonic	worst	best behaved
Chebyshev I	equiripple	monotonic	better	worse
Chebyshev II	monotonic	equiripple	better	worse
Elliptic	equiripple	equiripple	best	worst

Butterworth

When the passband must be flat and you can afford the order. No ripple anywhere; the roll-off is gentle.

Chebyshev I

Accept some passband ripple, get a steeper skirt. The ripple is equal across the whole passband.

Chebyshev II

The mirror: a clean passband and an equiripple stopband with transmission zeros in it. Useful when passband flatness matters more than stopband uniformity, and it has the gentlest phase of the two Chebyshevs.

Elliptic (Cauer)

Ripple in both bands, and the steepest transition any order can give. The order equation is a three-way trade: you cannot ask for order, ripple *and* transition width independently, since any two fix the third. Use it when you want the cheapest filter that meets a hard mask, and avoid it when phase or transient response matters.

With **Smallest order** ticked, remez tells you the lowest order that meets your specification. The **Result** panel reports both the order used and the estimate, so you can see how much margin you have.

5 The save file

Save design... writes a `.remz` file. It is UTF-8 JSON — the extension only changes what the desktop does with it, and anything that reads JSON still can. macOS declares it as `com.deadhat.remez.design` conforming to `public.json`, so a double-clicked design opens in `remez`.

The file records the *specification*, not the result: the coefficients are not in it, because opening the file redesigns the filter from the numbers you typed. That keeps a design portable between versions, and between this program and the Python prototype in `python_prototype/`, which reads and writes the same format.

5.1 A complete example

```
{
  "format": "remez-filter-design",
  "version": 1,
  "mode": "FIR - Remez exchange",
  "fs": 48000.0,
  "fir": {
    "numtaps": 41,
    "symmetry": "symmetric",
    "grid_density": 16,
    "maxiter": 60,
    "use_spec": false,
    "method": "remez",
    "window": "hamming",
    "kaiser_beta": "8.6",
    "half_band": false,
    "half_band_edge": "0.2",
    "rate_factor": 1,
    "rate_change": "decimate",
    "bands": [
      [
        0.0,
        0.2,
        1.0,
        1.0,
        1.0,
        0.5,
        false
      ],
      [
        0.25,
        0.5,
        0.0,
        0.0,
        10.0,
        50.0,
        false
      ]
    ]
  },
  "iir": {
    "response": "Lowpass",
    "approximation": "Butterworth",
    "order": 6,
    "auto_order": true,
    "wp": [
      "0.2",
```

```

        "0.4"
    ],
    "ws": [
        "0.3",
        "0.45"
    ],
    "rp": "0.5",
    "rs": "40"
},
"arithmetic": {
    "kind": "fixed",
    "word_bits": 16,
    "auto_frac": true,
    "frac_bits": -611,
    "headroom": 2,
    "fixed_coeffs": true,
    "structure": "tree",
    "folded": false,
    "testbench": true,
    "measure_noise": false,
    "sensitivity": false
},
"display": {
    "log_scale": true,
    "view": "Plot view",
    "appearance": "system",
    "phase": false,
    "signal": false,
    "signal_kind": "chirp",
    "signal_frequency": "0.05",
    "signal_length": 512,
    "group_delay": true,
    "zplane": true
}
}

```

Note: the `mode` value contains an em dash in the real file ("FIR — Remez exchange"); it is written as a hyphen above only so this listing stays plain ASCII.

5.2 Semantics

Top level.

key	meaning
<code>format</code>	Must be "remez-filter-design". A file without it is rejected.
<code>version</code>	Currently 1.
<code>mode</code>	"FIR -- Remez exchange" or "IIR -- bilinear transform". Only the leading FIR/IIR is significant to the reader.
<code>fs</code>	Sample rate. Every frequency in the file is in these units — the file does not normalise.

Both the `fir` and `iir` sections are always written, whichever mode is active, so switching mode after loading finds the other design where you left it.

fir. `bands` is an array of rows, each a fixed seven-element array:

index	0	1	2	3	4	5	6
field	f_start	f_stop	d_start	d_stop	weight	spec_dB	inverse_f

The first five and the seventh are what the table shows; `spec_dB` is used in place of `weight` when `use_spec` is true. `inverse_f` is a boolean. `grid_density` and `maxiter` are the search parameters described above, and `symmetry` is "symmetric" or "antisymmetric".

`method` is "remez", "leastSquares" or "window"; `window` is "rectangular", "hann", "hamming", "blackman" or "kaiser", and `kaiser_beta` is a string. `half_band` is a boolean and `half_band_edge` the one edge it needs. `rate_factor` is an integer, 1 for no rate change, and `rate_change` is "decimate" or "interpolate".

iir. `response` and `approximation` are stored as the *display* spellings — "Lowpass", "Chebyshev I" — because that is what the Python prototype writes. The reader accepts either those or the internal keys (`lowpass`, `chebyshev1`).

`wp` and `ws` are arrays of two edge values as strings, even for a lowpass, where only the first is used. `rp` and `rs` are the ripple and attenuation in dB. `order` applies when `auto_order` is false.

arithmetic. `kind` is "float" or "fixed". The rest are the Arithmetic panel's fields: `word_bits`, `auto_frac`, `frac_bits`, `headroom`, and the hardware options `fixed_coeffs`, `structure` ("chain", "tree" or "mac"), `folded`, `testbench` and `module_name`. `measure_noise` and `sensitivity` switch on the two analyses.

display. `log_scale`, `view` ("Plot view" or "Design view") and `appearance` ("system", "light" or "dark"), plus which optional plots are drawn — `phase`, `group_delay` and `zplane`, all booleans — and the signal runner's `signal`, `signal_kind` ("impulse", "step", "chirp", "tone", "noise" or "square"), `signal_frequency` and `signal_length`. Of those, only `log_scale` and `view` are keys the Python prototype writes.

5.3 Compatibility

Reading is deliberately forgiving.

- **Anything missing keeps its current value.** A file with only `format` and `version` loads and changes nothing.
- **Anything unrecognised is preserved.** The Python prototype writes `show_ext`, `show_noise`, `show_spec` and `folded_panels`, which this program has no controls for. They are carried through a load-and-save round trip rather than dropped, so editing a design here does not strip settings the other program relies on. `appearance` is this program's own addition and the Python's reader ignores it in the same way.
- **A file that is not a design is refused** rather than half-loaded.

Figures generated by `test/make_docs_test.dart`; rerun it with `--update-goldens` after changing the interface.